

```

import numpy as np

np.random.seed(42)

data = np.array([
    1.2, 1.5, 1.8, 2.0, 2.2,
    2.4, 2.6, 2.8, 3.0, 3.2,
    8.1, 8.3, 8.5, 8.7, 9.0,
    9.2, 9.4, 9.6, 9.8, 10.0
])

k = 2
n = len(data)

means = np.array([2.0, 9.0])
variances = np.array([1.0, 1.0])
weights = np.array([0.5, 0.5])

max_iterations = 100
tolerance = 0.000001

def gaussian_probability(x, mean, variance):
    coefficient = 1 / np.sqrt(2 * np.pi * variance)
    exponent = np.exp(-((x - mean) ** 2) / (2 * variance))
    return coefficient * exponent

for iteration in range(max_iterations):

    old_means = means.copy()
    old_variances = variances.copy()
    old_weights = weights.copy()

    responsibilities = np.zeros((n, k))

    for i in range(n):
        for j in range(k):
            probability = gaussian_probability(
                data[i],
                means[j],
                variances[j]
            )

            responsibilities[i][j] = weights[j] * probability

```

```

total = np.sum(responsibilities[i])

if total != 0:
    responsibilities[i] /= total

for j in range(k):

    responsibility_sum = np.sum(responsibilities[:, j])

    if responsibility_sum != 0:

        means[j] = np.sum(
            responsibilities[:, j] * data
        ) / responsibility_sum

        variances[j] = np.sum(
            responsibilities[:, j] *
            (data - means[j]) ** 2
        ) / responsibility_sum

        weights[j] = responsibility_sum / n

    mean_change = np.max(np.abs(means - old_means))
    variance_change = np.max(
        np.abs(variances - old_variances)
    )
    weight_change = np.max(
        np.abs(weights - old_weights)
    )

    total_change = max(
        mean_change,
        variance_change,
        weight_change
    )

    if total_change < tolerance:
        break

cluster_labels = np.argmax(
    responsibilities,
    axis=1
)

```

```

print("Expectation-Maximization Algorithm")
print("-----")

print("\nNumber of Iterations:", iteration + 1)

print("\nFinal Means:")
for i in range(k):
    print("Cluster", i + 1, ":", round(means[i], 4))

print("\nFinal Variances:")
for i in range(k):
    print("Cluster", i + 1, ":", round(variances[i], 4))

print("\nFinal Weights:")
for i in range(k):
    print("Cluster", i + 1, ":", round(weights[i], 4))

print("\nResponsibilities:")
print(responsibilities)

print("\nCluster Assignments:")

for i in range(n):
    print(
        "Data:",
        data[i],
        "-> Cluster",
        cluster_labels[i] + 1
    )

print("\nClustered Data:")

for cluster in range(k):
    print(
        "Cluster",
        cluster + 1,
        ":",
        data[cluster_labels == cluster]
    )

print("\nFinal Results:")
print("Cluster 1 Mean =", round(means[0], 4))
print("Cluster 2 Mean =", round(means[1], 4))
print("EM Algorithm Converged Successfully")

```

```

-----

... Number of Iterations: 2

Final Means:
Cluster 1 : 2.27
Cluster 2 : 9.06

Final Variances:
Cluster 1 : 0.3841
Cluster 2 : 0.3804

Final Weights:
Cluster 1 : 0.5
Cluster 2 : 0.5

Responsibilities:
[[1.00000000e+00 2.41649008e-35]
 [1.00000000e+00 5.15010565e-33]]

Data: 9.2 -> Cluster 2
Data: 9.4 -> Cluster 2
... Data: 9.6 -> Cluster 2
Data: 9.8 -> Cluster 2
Data: 10.0 -> Cluster 2

Clustered Data:
Cluster 1 : [1.2 1.5 1.8 2.  2.2 2.4 2.6 2.8 3.  3.2]
Cluster 2 : [ 8.1  8.3  8.5  8.7  9.   9.2  9.4  9.6  9.8 10. ]

Final Results:
Cluster 1 Mean = 2.27
Cluster 2 Mean = 9.06
EM Algorithm Converged Successfully

```